

CS 3006
Parallel and Distributed Computing
Lecture 3B

Danyal Farhat
FAST School of Computing
NUCES Lahore

Four Laws of Parallel Performance

Lecture's Agenda

- **Why Parallel Performance Laws?**
- **Amdahl's law — the pessimist**
- **Gustafson–Barsis Law — the optimist**
- **Amdahl's and Gustafson–Barsis Laws Comparison**
- **Karp-Flatt Metric**
- **The roofline model — the physicist**
- **Summary of the four laws**

Lecture's Agenda

- **Types of Parallelism**
 - Data Parallelism
 - Functional Parallelism
 - Pipelining
- **Multi-Processor vs. Multi-Computer**
- **Cluster vs. Network of Workstations**
- **Cache Coherence, Snooping, and Branch Prediction**
- **Summary**
- **Additional Resources**

Why Parallel Performance Laws?

- You are the CTO. A team asks to rent **100 more cloud processors** to speed up a data pipeline. Do you sign?
- You cannot answer that by intuition. You need a model that predicts what 100 processors will buy, a way to measure whether you got it, and a diagnosis when you did not.
- This lecture gives you all three — plus a fourth tool that tells you whether the bottleneck was ever the processors at all.

Why Parallel Performance Laws? (Cont.)

Tool	Kind	Question it answers	You must know...
Amdahl's law	prediction	"This problem, fixed. How much faster with N processors?"	the serial fraction of the 1-processor run
Gustafson-Barsis	prediction	"Same time budget. How much <i>bigger</i> a problem with N ?"	the serial fraction of the N -processor run
Karp-Flatt	diagnosis	"I measured a disappointing speedup. <i>Why?</i> "	nothing — you measure S and it tells you
Roofline	diagnosis & bound	"Am I limited by the processor or by memory?"	FLOPs and bytes moved by your kernel

Why Parallel Performance Laws?

Strong Scaling and Weak Scaling:

Strong scaling

Problem size W is **fixed**. Add processors, measure how the time falls.

Governed by: Amdahl.

Asks: how fast can I solve *this*?

Typical of: latency-critical work — a web request, a trading decision, tonight's forecast.

Weak scaling

Work *per processor* is fixed, so W grows with N . Measure whether the time stays constant.

Governed by: Gustafson–Barsis.

Asks: how big a problem can I afford?

Typical of: throughput work — simulation resolution, model size, dataset scale.

Amdahl's law — the pessimist

- Amdahl's law was **formulized** in 1967
- It shows an **upper-bound** on the **maximum speedup** that can be achieved
- Suppose you are going to design a **parallel algorithm** for a problem
- Further suppose that fraction of total time that the algorithm must consume in **serial** executions is 'F'
- This implies fraction of **parallel** portion is (1- F)
- Now, Amdahl's law **states** that

$$\text{Speedup}(p) = \frac{1}{F + \frac{1-F}{P}}$$

- Here 'p' is total number of available **processing** nodes

Amdahl's law — the pessimist (Cont.)

Derivation:

- Let's suppose you have a sequential code for a problem that can be executed in total $T(s)$ time
- $T(p)$ be the parallel time for the same algorithm over p processors.
- Then speedup can be calculated using:

$$\text{Speedup}(p) = \frac{T(s)}{T(p)}$$

- $T(p)$ can be calculated as:

$$T(p) = \text{serial comput. time} + \text{Parallel comp. time}$$
$$T(p) = F \cdot T(s) + \frac{(1-F) \cdot T(s)}{P}$$

Amdahl's law — the pessimist (Cont.)

Derivation:

- Again

$$\text{Speedup}(p) = \frac{T(s)}{T(p)} \Rightarrow \frac{T(s)}{F \cdot T(s) + \frac{(1-F) \cdot T(s)}{P}}$$

$$\Rightarrow \text{Speedup}(p) = \frac{1}{F + \frac{1-F}{P}}$$

- What if you have infinite number of processors?

Amdahl's law — the pessimist (Cont.)

- **Example:** Suppose we have a system with one processor and a serial program. We want to upgrade this system and we have the following two options.
 - a) Increase the number of processors from 1 to 8 and parallelize 40% of the code.
 - b) Increase the number of processors from 1 to 4 and parallelize 70% of the code.
- Calculate speedup in each case and identify which one is better?

Amdahl's law — the pessimist (Cont.)

Example 1.4.1 (The 5% that ruins everything). Suppose 95% of your program parallelises perfectly and 5% does not, so $f = 0.05$.

$$S(10) = \frac{1}{0.05 + \frac{0.95}{10}} = \frac{1}{0.145} = 6.9, \quad S(100) = \frac{1}{0.05 + 0.0095} = 16.8,$$

$$S(1000) = \frac{1}{0.05 + 0.00095} = 19.6, \quad S(\infty) = \frac{1}{0.05} = 20.$$

Going from 100 to 1000 processing elements — a tenfold increase in hardware cost and power — buys you 17% more speed. By the corollary, you were already halfway to the ceiling at $p = (1 - f)/f = 19$ processors.

Now note the efficiency. At $p = 1000$, $E = 19.6/1000 = 0.0196$. You are using about 2% of the machine you paid for.

Amdahl's law — the pessimist (Cont.)

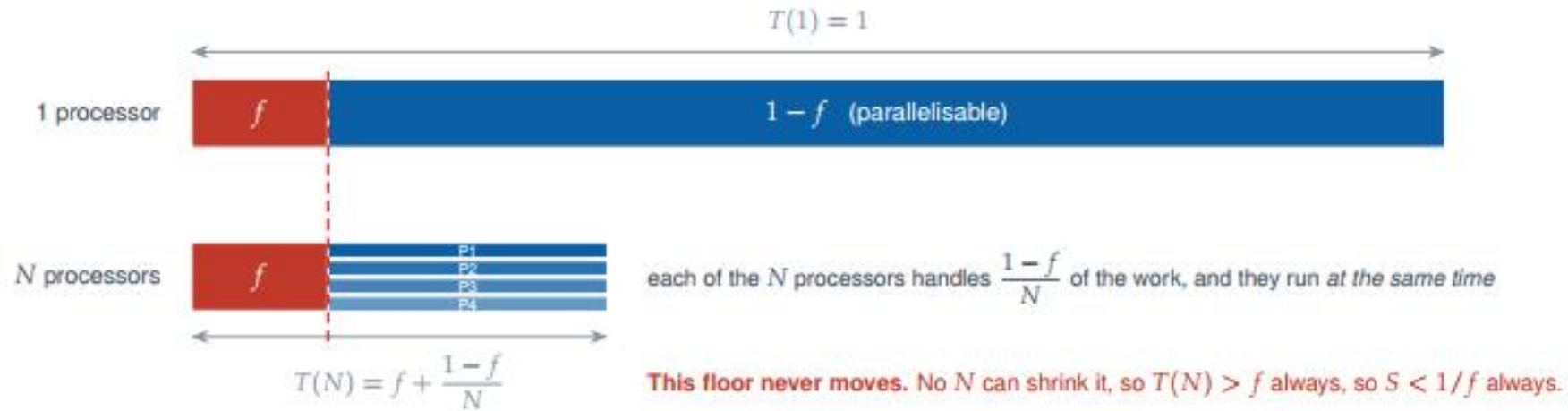


Figure 2. Amdahl's law as a picture. Normalise the one-processor time to 1. The serial fraction f is untouched by parallelism; only the remaining $1 - f$ is divided among N processors. The red dashed line is the hard floor on execution time, and everything else in this section is an algebraic consequence of its existence. Two conventions to note: the top bar is one continuous timeline, so the red and blue blocks abut exactly — there is no gap between them, because no time is unaccounted for. In the lower bar the four stacked slices are the *same* stretch of time seen on four processors at once, which is why the bar gets shorter rather than taller: they are concurrent, not consecutive. The figure is drawn for $N = 4$.

Amdahl's Law Assumptions

1. **Fixed problem size.** W does not change with N . This is the big one, and §3 is entirely about relaxing it.
2. **Perfect parallelisation.** The parallel portion divides into N equal pieces with no remainder and no load imbalance.
3. **Zero overhead.** No thread creation, no communication, no synchronisation, no cache-coherence traffic.
4. **Fixed serial fraction.** f does not depend on N .
5. **Identical processors.** All N are the same speed.

Key idea

Amdahl's law says: *optimise the serial fraction first*. A program with $f = 5\%$ can never exceed $20\times$, so on a 64-core machine there is far more value in halving f than in any amount of tuning of the parallel region. Before you rent 100 processors, measure f .

Gustafson–Barsis Law — the optimist

- John L. Gustafson and Edwin H. Barsis presented **Gustafson's law** in the article Reevaluating Amdahl's Law in 1988.
- Gustafson's law addresses the **shortcomings of Amdahl's law**, which is based on the assumption of a **fixed problem size**, that is of an execution workload that does not change with respect to the improvement of the resources.
- Gustafson's law instead proposes that programmers tend to **increase the size of problems** to fully exploit the computing power that becomes available as the resources improve.

Gustafson–Barsis Law — the optimist (Cont.)

- Gustafson estimated the speedup S of a program gained by using parallel computing as follows:

$$\begin{aligned} S &= s + p \times N \\ &= s + (1 - s) \times N \end{aligned}$$

- S is the **theoretical speedup** of the program with parallelism
- N is the **number of processors**
- s and p are the fractions of time spent executing the **serial parts** and the **parallel parts** of the program
- Gustafson observed from their workloads that time for the **serial part typically does not grow** as the problem and the system scale, i.e. s is fixed.

Gustafson–Barsis Law — the optimist (Cont.)

Example 1.4.2. With $s = 0.05$ — the same 5% sequential fraction that capped Amdahl's speedup at 20 — Gustafson gives

$$S_{\text{scaled}}(1000) = 0.05 + 1000(0.95) = 950.05.$$

The efficiency is $950/1000 = 0.95$, not 0.02. Same 5%, completely different conclusion.

Amdahl's and Gustafson–Barsis Laws Comparison

The two laws do not contradict each other

They answer different questions, and the difference is entirely in what is held fixed.

	Amdahl (1967)	Gustafson–Barsis (1988)
What is held fixed	the problem size	the run time you will tolerate
f or s measured against	the <i>serial</i> execution	the <i>parallel</i> execution
The question answered	“I have this job. How much faster can I finish it?”	“I have this much time. How much bigger a job can I do?”
Speedup behaviour	saturates at $1/f$	grows linearly, slope $1 - s$

Use Amdahl when the problem genuinely will not grow: a fixed-size simulation that must finish before a deadline, or a latency-critical request. Use Gustafson when it will: weather models get finer, neural networks get larger, datasets get bigger. Most of the reason the field did not die in 1967 is that the second case is far more common than the first.

Amdahl's and Gustafson–Barsis Laws Comparison

Key idea — the resolution

The two laws never disagree about the same experiment. They differ because they *describe different experiments*:

- Amdahl holds W fixed and lets time fall. Efficiency $\rightarrow 0$.
- Gustafson holds T fixed and lets W grow with N . Efficiency $\rightarrow 1 - \sigma$.

Gustafson's optimism comes entirely from the *assumption that the workload grows*, not from any different mathematics. If your problem genuinely cannot grow — a fixed-resolution image, a single fixed query, tonight's forecast for tonight's grid — then Gustafson's law does not rescue you, and Amdahl's ceiling is real and binding.

Karp-Flatt Metric

- Amdahl's law ignore the parallelization overhead due to which it **overestimate** the achievable speedup.
- The Karp - Flatt metric is used to calculate **serial fraction** for a given parallel configuration.
- Experimentally **determined** serial fraction e takes into account parallel overhead.
- Detects other **sources of overhead** or inefficiency ignored in speedup model
 - Process startup time
 - Process synchronization time
 - Imbalanced workload
 - Architectural overhead

Karp-Flatt Metric (Cont.)

- If a parallel program is exhibiting a **speedup S** while using **P processing units** then experimentally determined serial fraction e is given by :

$$e = \frac{1/S - 1/p}{1 - 1/p}$$

- Experimentally determined serial fraction e may either stay **constant** with respect to p (meaning that the parallelization overhead is **negligible**) or **increase** with respect to p (meaning that parallelization overhead **dominates** the speedup)

Karp-Flatt Metric (Cont.)

- **Proposed** by Alan H. Karp and Horace P. Flatt in 1990
- To **determine** to what extent an algorithm is parallelized
- If metric value remains **consistent** w.r.t. increasing number of processors and speedups, we need to reduce sequential fraction of the program
- If metric value **increases** as increasing number of processors and speedups, we need to reduce parallelize overheads

Karp-Flatt Metric – Example

- **Example 1:** Suppose in a parallel program, for the given processors, you gained a speedup given in second row, determine sequential fraction of your program.

Solution: Compute $e(n, p)$ corresponding to each data point:

p	2	3	4	5	6	7	8
S	1.82	2.50	3.08	3.57	4.00	4.38	4.71
$e(n, p)$	0.1	0.1	0.1	0.1	0.1	0.1	0.1

Since the experimentally determined serial fraction $e(n, p)$ is not increasing with p , the primary reason for the poor speedup is the 10% of the computation that is inherently sequential. Parallel overhead is not the reason for the poor speedup.

Karp-Flatt Metric – Example (Cont.)

- **Example 2:**

Benchmarking a parallel program on 1, 2, ..., 8 processors produces the following speedup results:

p	2	3	4	5	6	7	8
s	1.87	2.61	3.23	3.73	4.14	4.46	4.71

What is the primary reason for the parallel program achieving a speedup of 4.71 on 8 processors?

Solution:

p	2	3	4	5	6	7	8
s	1.87	2.61	3.23	3.73	4.14	4.46	4.71
e	0.07	0.075	0.08	0.085	0.09	0.095	0.1

Since the experimentally determined serial fraction e is steadily increasing with p , parallel overhead also contributes to the poor speedup.

Karp-Flatt Metric (Cont.)

Key idea

Speedup tells you *what* happened. The Karp-Flatt metric tells you *why*. It is not a prediction; it is a forensic tool you apply after the experiment.

e constant

Healthy. You are simply hitting Amdahl's limit. The lost efficiency is due to genuinely limited parallelism in the program, not to a bug.

Action: reduce the serial algorithm itself, or accept the ceiling.

e rising smoothly

Overhead growing. Synchronisation cost typically grows as $O(p)$ or $O(\log p)$, producing a smooth rise.

Action: coarsen the task granularity; batch messages; reduce barrier frequency.

e rising erratically

Load imbalance. Work divides evenly for some p and badly for others, so e jumps around.

Action: dynamic scheduling, work stealing, or a decomposition that divides evenly.

e falling

Aggregate cache effects. Extra processors brought extra cache and bandwidth, cutting per-processor overhead.

Action: none needed — but check your baseline is fair.

Figure 7. The four fingerprints of the Karp-Flatt metric. Note that speedup and efficiency curves cannot distinguish these cases: all four look like “less than linear scaling.” That is the entire value of the metric.

The roofline model — the physicist

- First three laws (Amdahl, Gustafson-Barsis, and Karp-Flatt) has been about **processor** count.
- The **roofline model**, introduced by Williams, Waterman and Patterson in 2009 [4], asks a question:
 - Is my program slow because the processor cannot compute fast enough, or because memory cannot supply data fast enough?
- This matters because the two diagnoses call for opposite responses.
 - If you are **compute-bound**, buying faster memory is money burned.
 - If you are **memory-bound**, buying a faster processor is money burned.

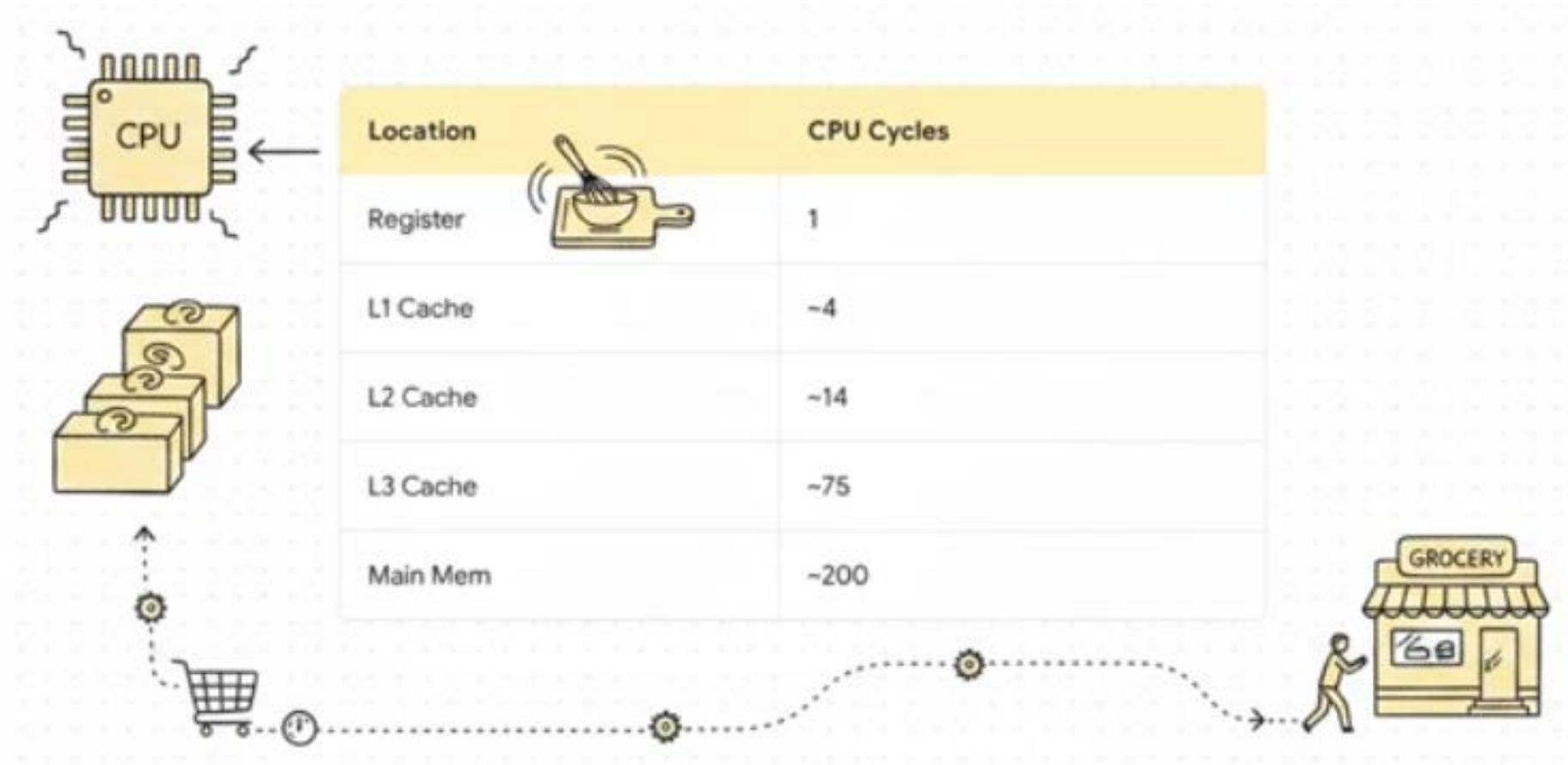
The roofline model — the physicist (Cont.)

Measuring Computing Speed:

- **FLOPS - Floating Point Operations Per Second done by a computer on a given algorithm.**
 - If a processor performs 100 floating point or arithmetic operations per second, we say that the processor's computing speed is 100 FLOPS.

The roofline model — the physicist (Cont.)

Computation vs Communication (Physical Data Movement):



The roofline model — the physicist (Cont.)

Calculating Arithmetic Intensity (Measuring Data Efficiency)

- Also known as Operational Intensity
- Arithmetic intensity I are the **math operations performed** per byte of data loaded or stored from memory.

$$I = \frac{\text{floating-point operations performed}}{\text{bytes moved between processor and memory}} \quad [\text{FLOP/byte}]$$

- Arithmetic intensity measures **how much useful arithmetic** you extract from each byte you were forced to fetch.
 - Low I means you are **hauling data** around and barely computing on it
 - High I means you fetch once and **compute a lot**

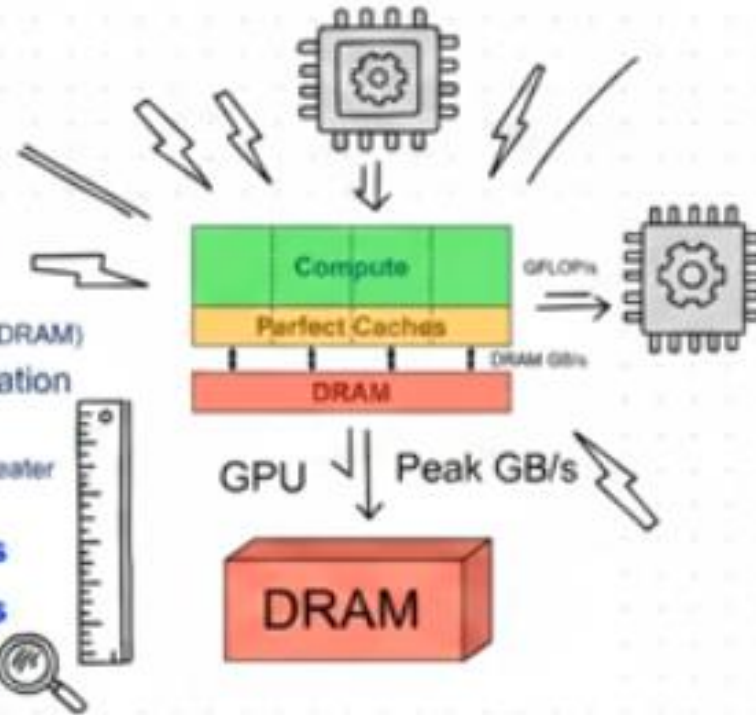
The roofline model — the physicist (Cont.)

Roofline Performance Model (Visualize Limits):

- Any given loop nest will perform:
 - Computation (e.g. FLOPs)
 - Communication (e.g. moving data to/from DRAM)
- With perfect overlap of communication and computation...
 - Run time is determined by whichever is greater

$$\text{GFLOP/s} = \min \begin{cases} \text{Peak GFLOP/s} \\ \text{AI} \cdot \text{Peak GB/s} \end{cases}$$

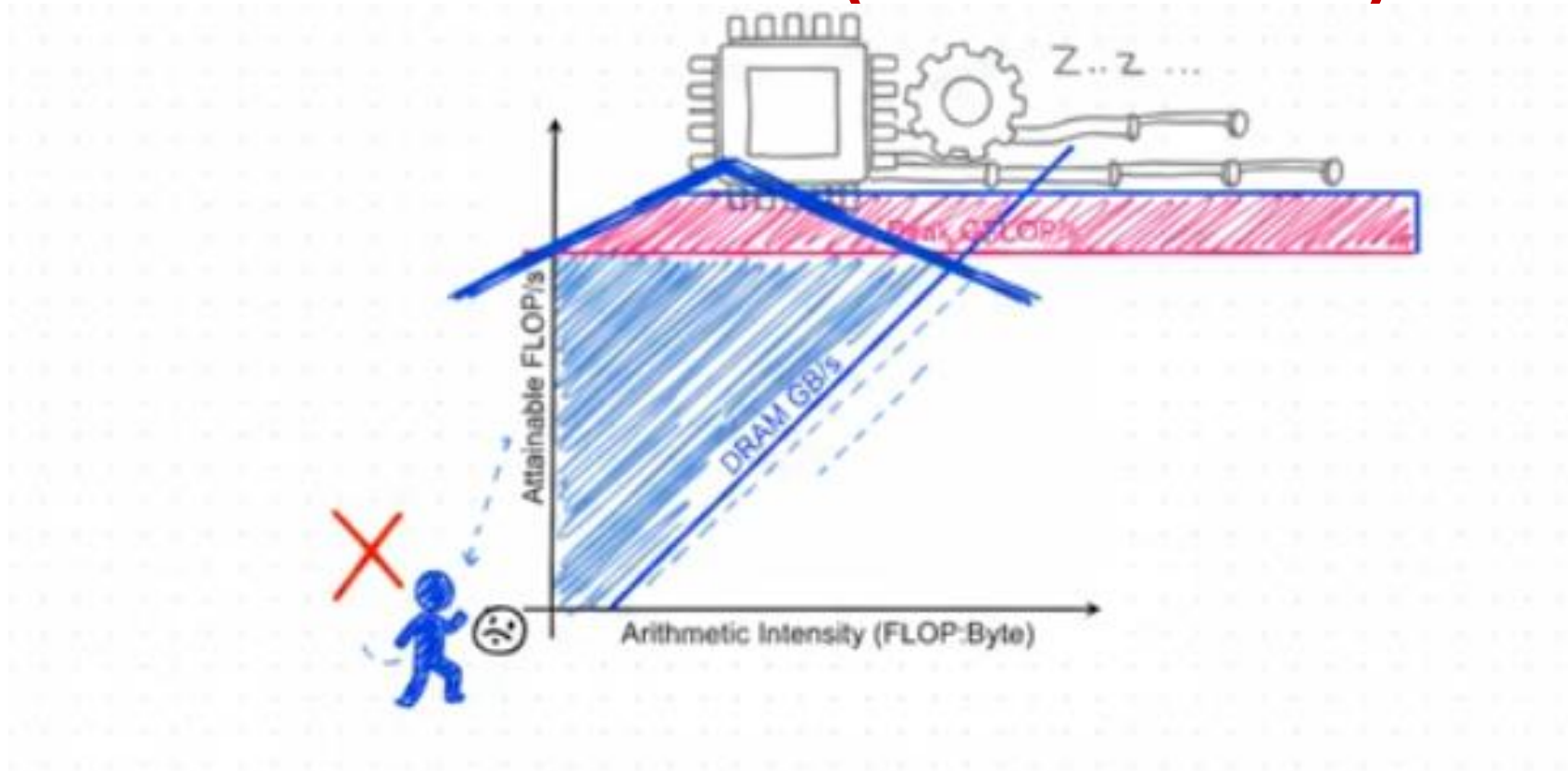
AI (Arithmetic Intensity) = FLOPs / Bytes (as presented to DRAM)



The **Roofline Equation** determining maximum execution speed.

The roofline model — the physicist (Cont.)

Roofline Performance Model (Visualize Limits):



Roofline Graph mapping FLOPs versus Arithmetic Intensity.

The roofline model — the physicist (Cont.)

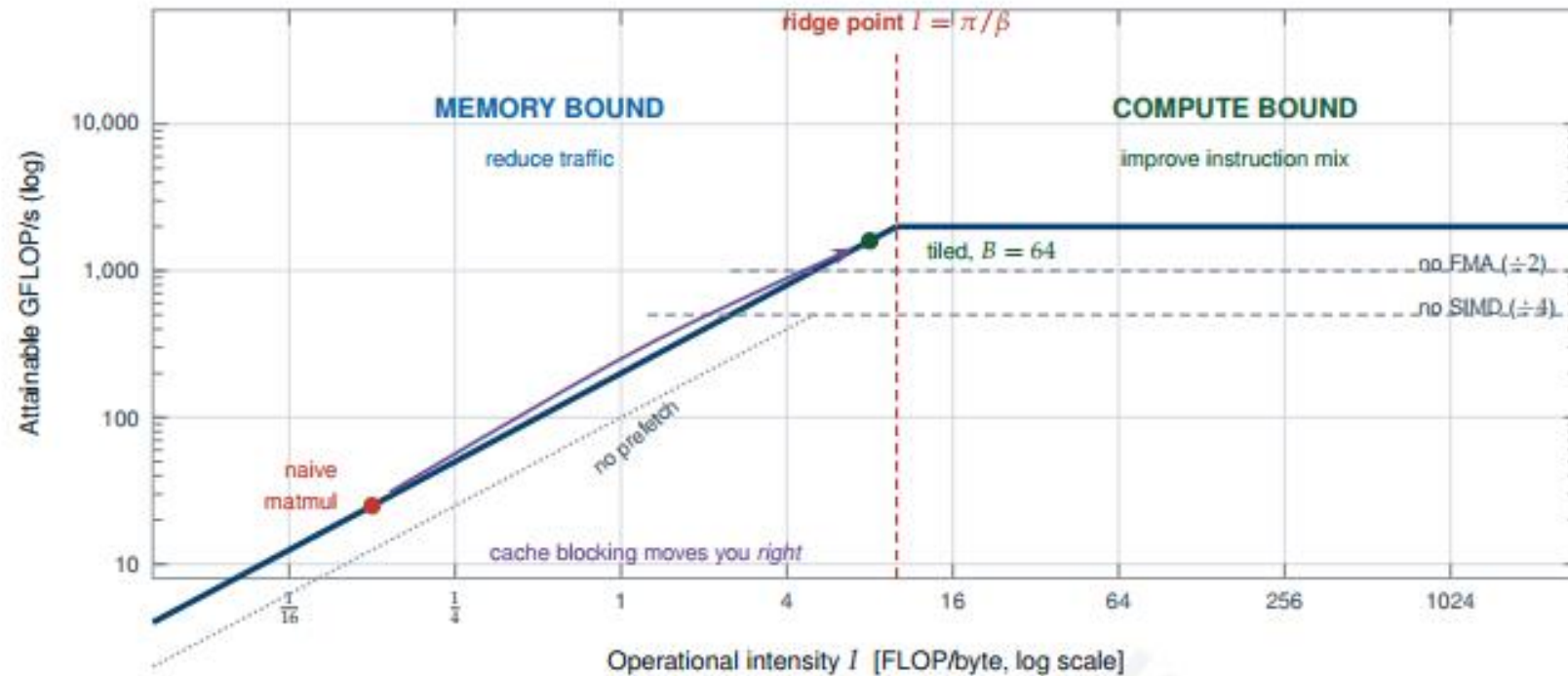


Figure 9. A roofline plot. The heavy line is the bound $P = \min(\pi, \beta I)$ for a machine with $\beta = 200$ GB/s and $\pi = 2$ TFLOP/s, so the ridge sits at $I = 10$ FLOP/byte. Dashed lines are *ceilings* — limits you hit if you fail to use FMA or SIMD, or if hardware prefetching is not engaged. The two dots show the same mathematical computation before and after cache blocking: identical FLOP count, $64\times$ less memory traffic, and a move from deep in the memory-bound region to just under the compute ceiling. Model after Williams, Waterman and Patterson [4].

The roofline model — the physicist (Cont.)

- In previous figure, Kernel K has an arithmetic intensity (I_{ridge}) of 10 FLOPs/byte.
- If your program's arithmetic intensity (I) is less than kernel's peak arithmetic intensity (I_{ridge}), program is **memory** bound.
 - If $I < I_{\text{ridge}}$ (i.e. $I < 10$ in previous figure), program is memory bound.
- If your program's arithmetic intensity (I) is greater than kernel's peak arithmetic intensity (I_{ridge}), program is **compute** bound.
 - If $I > I_{\text{ridge}}$ (i.e. $I > 10$ in previous figure), program is compute bound.

The roofline model — the physicist (Cont.)

- Most **modern kernels** are memory-bound, because their computation capacity is faster than their ability to fetch data from memory.

Using the model to choose an optimization:

	Memory-bound ($I < I_{\text{ridge}}$)	Compute-bound ($I > I_{\text{ridge}}$)
Symptom	CPU stalls waiting on data	Execution units saturated
Do this	cache blocking / tiling; improve data layout (AoS $\rightarrow$ SoA); fuse loops to reuse data in registers; compress or quantise; NUMA-aware allocation; software prefetch	vectorise (SIMD/AVX); use fused multiply-add; unroll to expose ILP; reduce instruction count; use lower precision where accuracy permits
Do NOT do this	buy a faster processor; add cores; hand-tune the assembly	buy faster memory; add memory channels

Summary of the four laws

	Formula	Limit	Tells you
Amdahl	$S = \frac{1}{f + \frac{1-f}{N}}$	$S \rightarrow 1/f; E \rightarrow 0$	the ceiling on a fixed problem
Gustafson	$S = N - \sigma(N - 1)$	$S \sim (1 - \sigma)N; E \rightarrow 1 - \sigma$	the payoff from a growing problem
Karp–Flatt	$e = \frac{N/S - 1}{N - 1}$	e constant if healthy	whether the loss is algorithm or overhead
Roofline	$P = \min(\pi, \beta I)$	ridge at $I = \pi/\beta$	whether to fix compute or memory

Summary of the four laws (Cont.)

The lecture in one paragraph

Amdahl says a fixed problem has a hard ceiling set by its serial fraction, and that ceiling arrives faster than anyone expects. Gustafson says the ceiling is a consequence of holding the problem fixed, and that if the problem grows with the machine, efficiency stabilises instead of collapsing — the same algebra, a different experiment. Karp–Flatt turns the first law around: measure the speedup, recover the serial fraction, and read its trend to distinguish an honest algorithmic limit from overhead you accidentally created. And the roofline steps outside the processor-count question entirely to ask whether you were ever compute-limited at all — which, in 2026, you usually were not.

Types of Parallelism

Data Parallelism

- When there are independent tasks applying the **same operation** to different elements of a data set
- Example code

```
for i=0 to 99 do
  a[ i ] = b[ i ] + c [ i ]
Endfor
```
- Here **same operation** addition is being performed on first 100 elements of 'b' and 'c' arrays
- All 100 iterations of the loop could be **executed simultaneously**

Types of Parallelism (Cont.)

Functional Parallelism

- When there are independent tasks applying **different operations** to different data elements
- Example code
 1. $a=2$
 2. $b=3$
 3. $m = (a+b) / 2$
 4. $s = (a \times a + b \times b) / 2$
 5. $v = s - m^2$
- Here third and fourth statements could be performed **concurrently**

Types of Parallelism (Cont.)

Pipelining

- Usually used for the problems where **single instance** of the problem can not be parallelized
- The output of **one stage** is input of the other stage
- Dividing whole computation of each instance into **multiple stages** provided that there are multiple instances of the problem

Types of Parallelism (Cont.)

Pipelining (Cont.)

- An effective method of **attaining parallelism** on the uniprocessor architectures
- Depends on **pipelining abilities** of the processor
 - Pentium 2 supports **12 stage** pipelining
 - Pentium 4 supports **20 stage** pipelining at 2.0 GHz

Types of Parallelism (Cont.)

- **Pipelining Example: Assembly line analogy**

Time	Engine	Doors	Wheels	Paint
5 min	Car 1			
10 min		Car 1		
15 min			Car 1	
20 min				Car 1
25 min	Car 2			
30 min		Car 2		
35 min			Car 2	
40 min				Car 2

Sequential Execution

Types of Parallelism (Cont.)

- **Pipelining Example: Assembly line analogy**

Time	Engine	Doors	Wheels	Paint
5 min	Car 1			
10 min	Car 2	Car 1		
15 min	Car 3	Car 2	Car 1	
20 min	Car 4	Car 3	Car 2	Car 1
25 min		Car 4	Car 3	Car 2
30 min			Car 4	Car 3
35 min				Car 4

Pipelining

Types of Parallelism (Cont.)

- **Pipelining Example: Assembly line analogy**

Cycles	Fetch	Decode	Execute	Save
1	Inst 1			
2	Inst 2	Inst 1		
3	Inst 3	Inst 2	Inst 1	
4	Inst 4	Inst 3	Inst 2	Inst 1
5		Inst 4	Inst 3	Inst 2
6			Inst 4	Inst 3
7				Inst 4

4-stage Pipelining

Multi-Processor vs. Multi-Computer

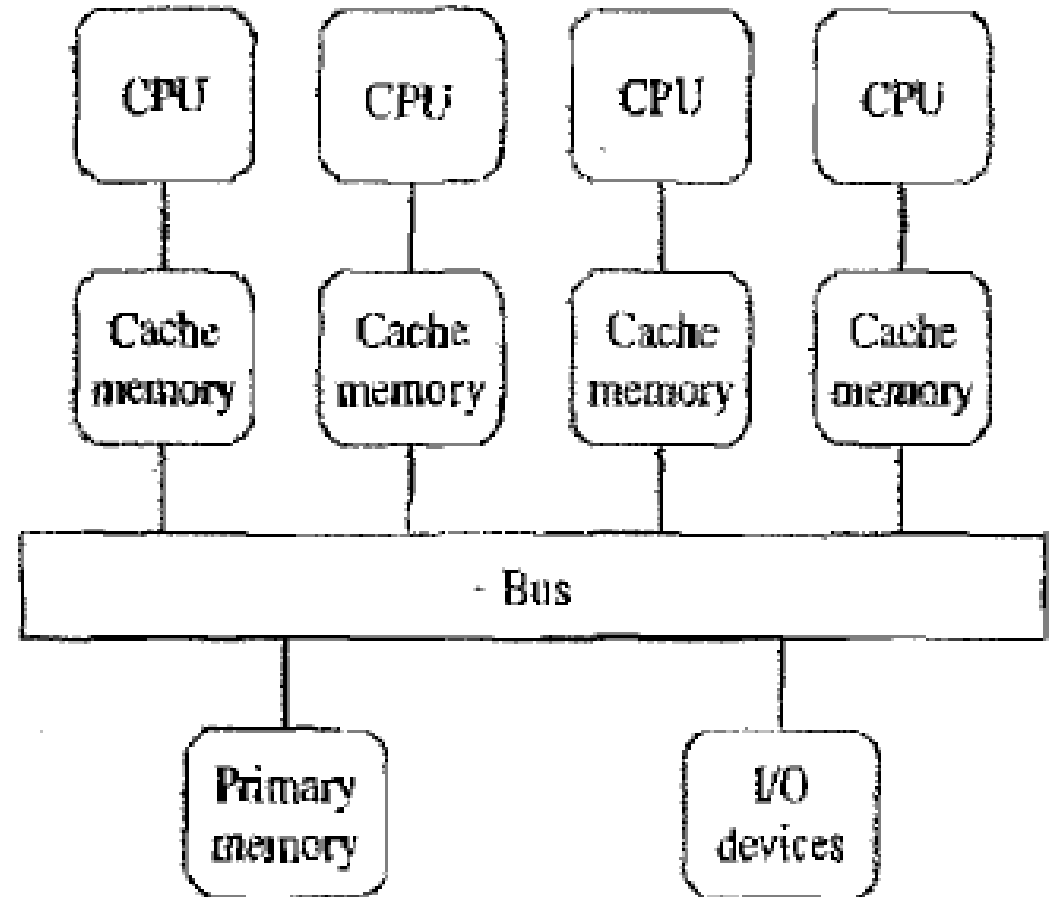
Multi-Processor

- Multiple **CPUs** with a shared memory
- Same address on two different CPUs **refers** to the same memory location
- Generally two categories
 - Centralized Multi-processor
 - Distributed Multi-processor

Multi-Processor vs. Multi-Computer (Cont.)

Centralized Multi-Processor

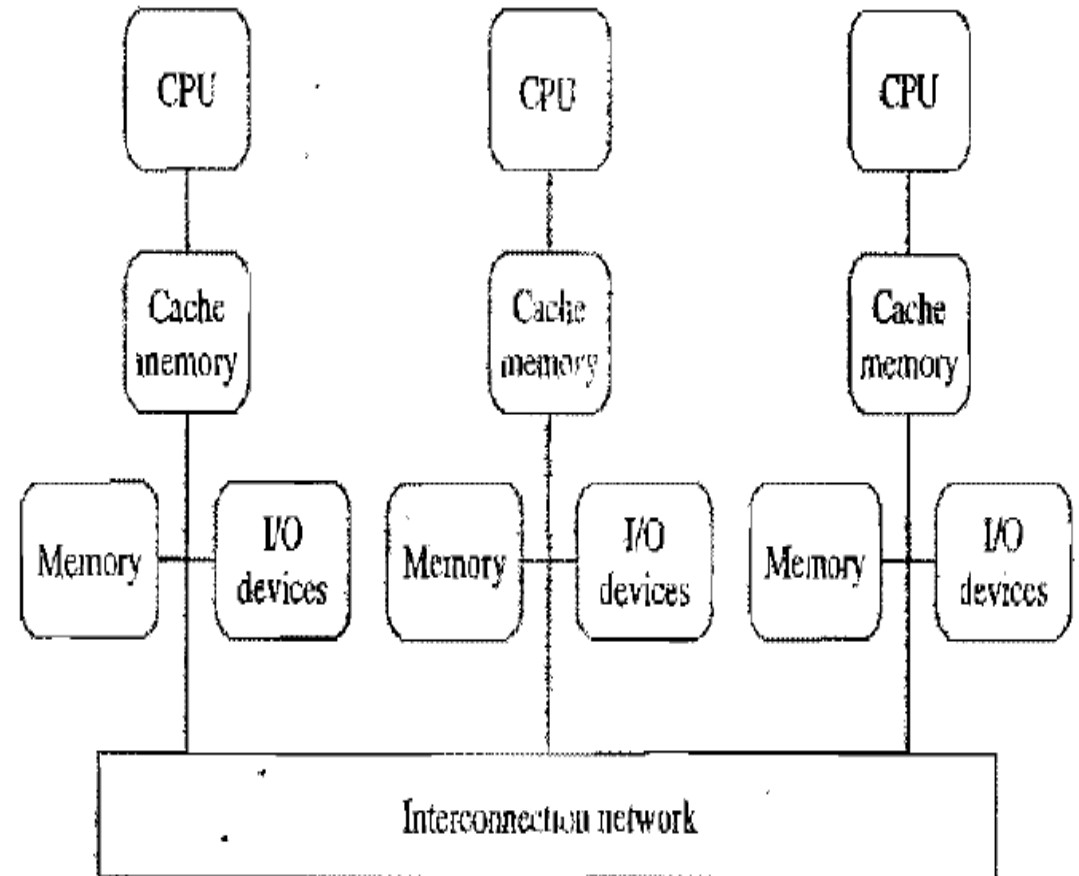
- Additional CPUs are **attached** to the system bus, and all the processors share the same primary memory
- All the memory is at **one place** and has the same access time from every processor
- Also known to as UMA (Uniform Memory Access) Multi-processor or SMP (Symmetrical Multi-processor)



Multi-Processor vs. Multi-Computer (Cont.)

Distributed Multi-processor

- Distributed collection of memories **forms** one logical address space
- Again, the same address on different processors **refers** to the same memory location
- Also known as Non-Uniform Memory Access (NUMA) architecture
- Because, memory access time **varies** significantly, depending on the physical location of the referenced address



Multi-Processor vs. Multi-Computer (Cont.)

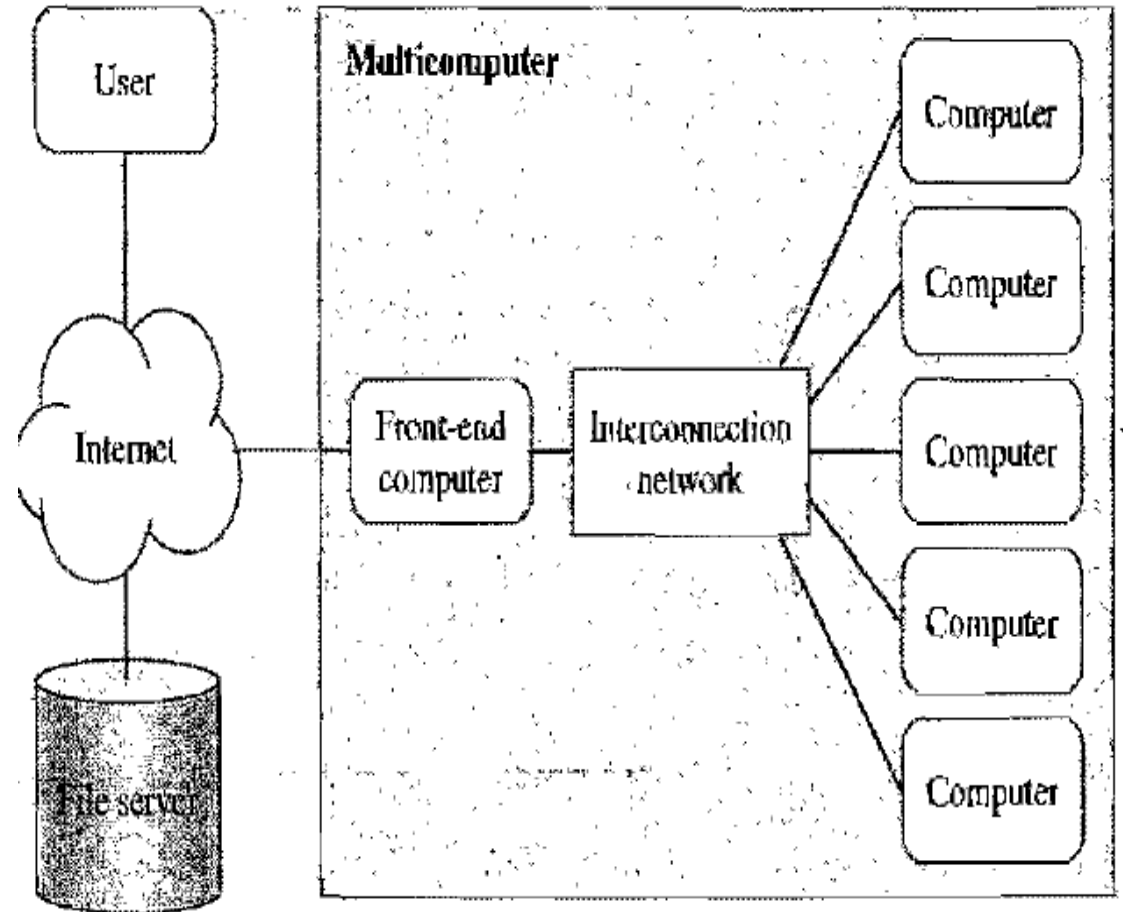
Multi-Computer

- Distributed memory, **multi-CPU** computer
- Unlike NUMA architecture, a multicomputer has **disjoint** local address spaces
- Each processor has **direct access** to their local memory only
- Same address on different processors **refers** to two different physical memory locations
- Processors **interact** with each other through passing messages

Multi-Processor vs. Multi-Computer (Cont.)

Asymmetric Multi-Computers

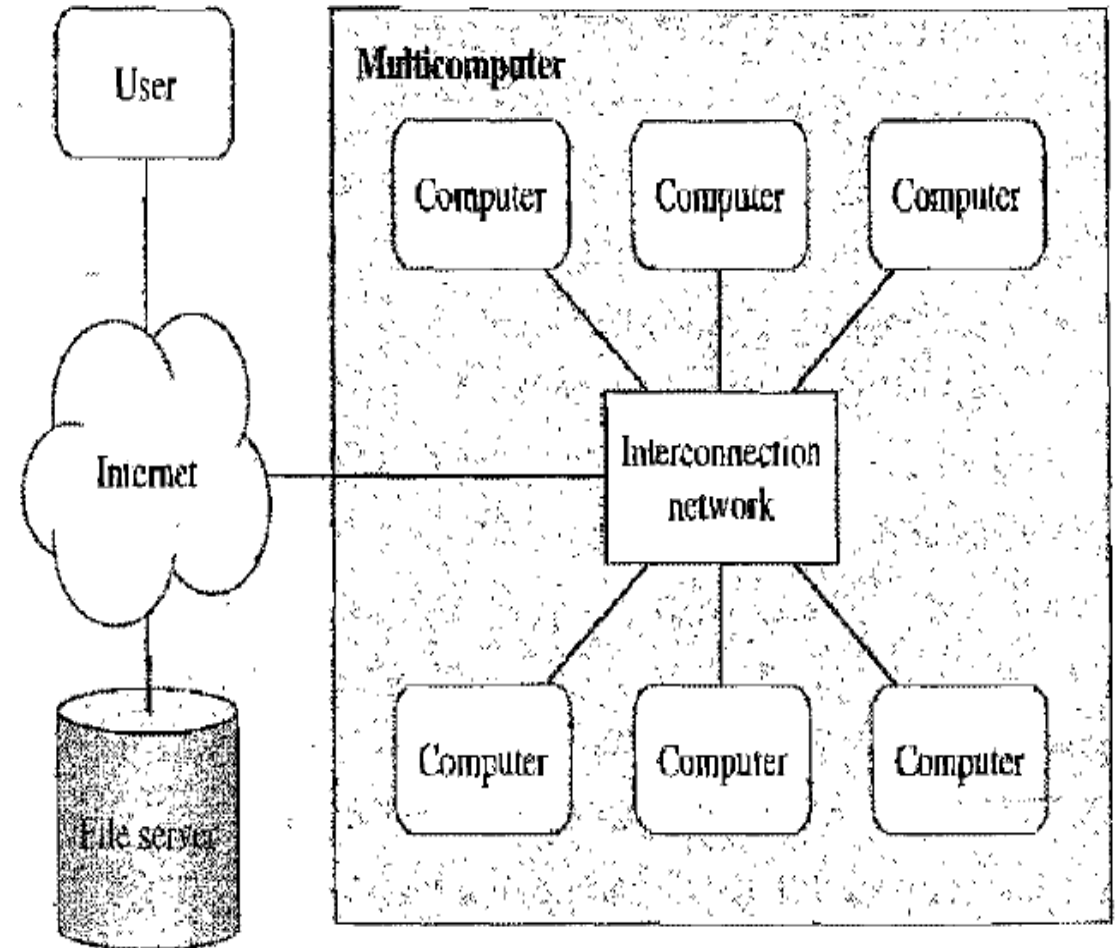
- A **front-end** computer that interacts with users and I/O devices
- The **back-end** processors are dedicatedly used for “number crunching”
- Front-end computer executes a full, multi-programmed OS and **provides** all functions needed for program development
- The backends are **reserved** for executing parallel programs



Multi-Processor vs. Multi-Computer (Cont.)

Symmetric Multi-Computers

- Every computer **executes** same OS
- Users may **log** into any of the computers
- This **enables** multiple users to concurrently login, edit and compile their programs
- All the nodes can **participate** in execution of a parallel program



Cluster vs. Network of Workstations

Cluster

- Usually a **co-located** collection of low-cost computers and switches, dedicated to running parallel jobs
- All computer run the **same version** of operating system
- Some of the computers may not have **interfaces** for the users to login
- Commodity cluster uses **high speed networks** for communication such as fast Ethernet at 100Mbps etc.

Network of workstations

- A **dispersed collection** of computers
- Individual workstations may have **different Operating systems** and executable programs
- Users have the power to **login and power off** their workstations
- Ethernet speed for this network is usually **slower**.
 - Typical in **range** of 10s Mbps

Cache Coherence

- When we are in a distributed environment, each CPU's cache needs to be consistent (continuously needs to be **updated** for current values), which is known as cache coherence

Snooping

- Snoopy protocols achieve **data consistency** between the cache memory and the shared memory through a bus-based memory system
- Write-invalidate and write-update **policies** are used for maintaining cache consistency

Snooping Protocol Types

Write-invalidate (mostly used)

- The processor that is writing data causes copies in the caches of all other processors in the system to be **rendered invalid** before it changes its local copy

Write-update

- The processor that is writing the data **broadcasts** the new data over the bus
- All caches that contain copies of the data are then **updated**

Branch Prediction

- Branch prediction is a technique used in CPU design that attempts to **guess the outcome** of a conditional operation and prepare for the most likely result
- Issues in pipelining of branch prediction:
 - If the **prediction is true** then the pipeline will not be flushed and no clock cycles will be lost
 - If the **prediction is false** then the pipeline is flushed and starts over with the current instruction

Summary

- **Why Parallel Performance Laws?**

- You are the CTO. A team asks to rent 100 more cloud processors to speed up a data pipeline. Do you sign?

- **Amdahl's law — the pessimist**

- It shows an upper-bound on the maximum speedup that can be achieved by increasing number of processors

Summary (Cont.)

- **Gustafson–Barsis Law — the optimist**

- Addresses the shortcomings of Amdahl's law, which is based on the assumption of a fixed problem size

- **Amdahl's and Gustafson–Barsis Laws Comparison**

- Amdahl holds W fixed and lets time fall.
- Gustafson holds T fixed and lets W grow with N .

Summary (Cont.)

- **Karp-Flatt Metric**

- Calculate serial fraction for a given parallel configuration
- Detects sources of overhead or inefficiency ignored in the speedup model
 - ✓ Process startup time, Process synchronization time
 - ✓ Imbalanced workload, Architectural overhead

$$e = \frac{\frac{1}{s} - \frac{1}{p}}{1 - \frac{1}{p}}$$

- e is constant with respect to p - parallelization overhead is negligible
- e increases with respect to p - parallelization overhead dominates the speedup

Summary (Cont.)

- The roofline model — the physicist

- Asks a question: “Is my program slow because the processor cannot compute fast enough, or because memory cannot supply data fast enough?”

$$I = \frac{\text{floating-point operations performed}}{\text{bytes moved between processor and memory}} \quad [\text{FLOP/byte}]$$

$$\text{GFLOP/s} = \min \begin{cases} \text{Peak GFLOP/s} \\ AI * \text{Peak GB/s} \end{cases}$$

$$P = \min(\pi, \beta I)$$

Summary (Cont.)

- **Types of Parallelism**

- **Data Parallelism**

- ✓ Independent tasks applying the same operation to different elements of a data set

- **Functional Parallelism**

- ✓ Independent tasks applying different operations to different data elements

- **Pipelining**

- ✓ Effective method of attaining parallelism on the uniprocessor architectures

Summary (Cont.)

- **Multi-Processor vs. Multi-Computer**

- **Multi-Processor - Multiple CPUs with a shared memory**

- ✓ Centralized Multi-Processor - All the memory is at one place and has Uniform Memory Access (UMA) architecture
 - ✓ Distributed Multi-processor - Distributed collection of memories forms one logical address space. Use Non-Uniform Memory Access (NUMA) architecture

- **Multi-Computer - Each processor has disjoint memory**

- ✓ Asymmetric Multi-Computers
 - A front-end computer interacts with users and I/O devices
 - The back-end processors are dedicated for processing
 - ✓ Symmetric Multi-Computers
 - Every computer executes same OS
 - This enables multiple users to concurrently login, edit and compile their programs

Summary (Cont.)

- **Cluster vs. Network of Workstations**

- **Cluster** - Co-located collection of computer with same version of OS
- **Network of Workstations** - Dispersed collection of computers with different versions of OS

- **Cache Coherence**

- **Cache** continuously needs to be updated for current values

Summary (Cont.)

- **Snooping**

- Snoopy protocols achieve data consistency between cache and memory through a bus based memory system
- Write invalidate & write update policies are used

- **Branch Prediction**

- Used in CPU design to guess the outcome of a conditional operation and prepare for the most likely result

Additional Resources

- **Compiled Notes on Four Laws of Parallel Performance**
- **Roofline Model Explained | GPU Course Part 16**
 - <https://www.youtube.com/watch?v=3U5t-3uaCew>

Questions?

Class Activity